



**INSTITUTO FEDERAL DE MATO GROSSO
CAMPUS CUIABÁ CEL. OCTAYDE JORGE DA
SILVA**

Daniel Barros

Gustavo Curado

**Projeto Final da Disciplina Transmissão e Comunicação de
Dados: Transmissão de dados confiável via rádio**

CUIABÁ - MT

2026

Introdução

Este projeto tem como objetivo o desenvolvimento de um sistema de comunicação sem fio para transmissão de textos e imagens monocromáticas de baixa resolução entre dois microcontroladores ESP32 utilizando módulos de rádio frequência de 433 MHz FS1000A. O sistema é composto por um transmissor e um receptor, sendo que o receptor utiliza um display TFT LCD ST7735 de 1,8 polegadas para exibição das informações recebidas.

Devido às limitações de largura de banda e confiabilidade inerentes à comunicação por rádio frequência, os dados são divididos em quadros de tamanho reduzido antes da transmissão. Para garantir a integridade das informações, foi implementado um mecanismo de detecção de erros baseado no algoritmo CRC (Cyclic Redundancy Check), permitindo identificar possíveis falhas ocorridas durante o envio dos dados. Além disso, é utilizado o protocolo Stop-and-Wait, responsável pelo controle de fluxo e confirmação da recepção dos quadros, aumentando a confiabilidade da comunicação.

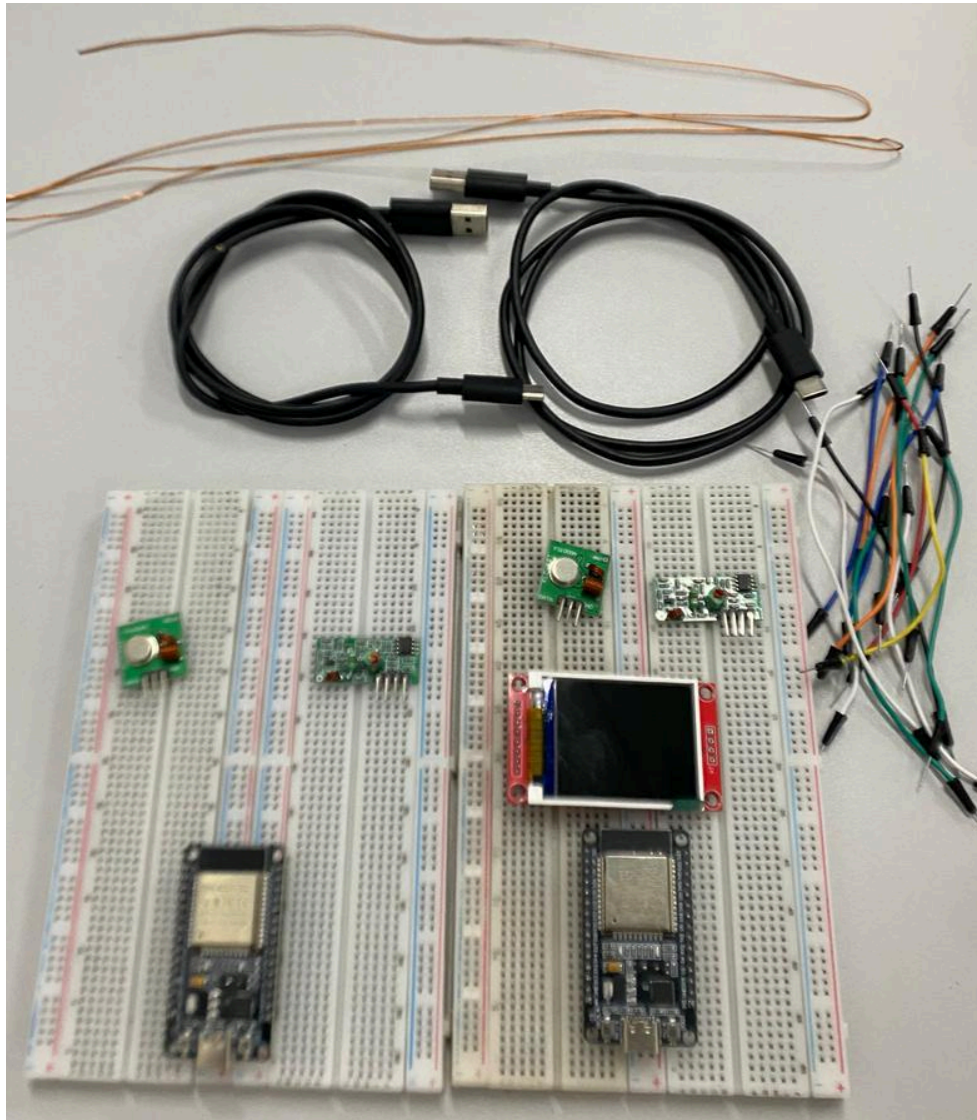
O sistema foi projetado para transmitir mensagens de texto e imagens monocromáticas de até 64×64 pixels, demonstrando a viabilidade da utilização de módulos de baixo custo em aplicações que exigem transferência de dados entre dispositivos embarcados. Os resultados esperados incluem a recepção correta dos dados transmitidos, a identificação de erros de comunicação e a exibição das informações no display do dispositivo receptor.

Metodologia e planejamento

Para a montagem do sistema foram utilizados os seguintes itens:

- 2 microcontroladores ESP32 (módulo de 30 pinos)
- 2 módulos transmissores RF FS1000A (433 MHz, modulação ASK/OOK)
- 2 módulos receptores RF XY-MK-5V (433 MHz)
- 1 display TFT LCD de 1,8" com controlador ST7735
- 4 placas de ensaio (protoboards) de 400 pontos
- Jumpers macho-macho e macho-fêmea em quantidade suficiente

- Fio de cobre rígido esmaltado para as antenas (≈ 17 cm cada)



Componentes utilizados. Fonte: autoria própria

Descrição dos componentes

ESP32 – Responsável por toda a lógica do sistema: geração e decodificação dos quadros, cálculo do CRC, execução do protocolo Stop-and-Wait e controle do display.

FS1000A – Transmissor de rádio-frequência. Converte o sinal digital (bit) em

portadora de 433 MHz por modulação OOK (On-Off Keying). Tem um pino DATA que aceita níveis lógicos de 3,3 V a 5 V e um pad para soldar a antena.

XY-MK-5V – Recetor super-heterodino. Recebe o sinal de 433 MHz e apresenta no pino DATA um sinal digital correspondente ao bit transmitido. Inclui um comparador com histerese, por isso o sinal de saída é já “limpo”, mas gera ruído aleatório na ausência de portadora.

Display ST7735 – Visor TFT de 128×160 píxeis, comunicação SPI. Usado para apresentar as mensagens de texto ou as imagens monocromáticas recebidas.

Antenas – Fios de 17,3 cm (1/4 de onda) soldados diretamente nos pads ANT dos módulos, fundamentais para irradiar e captar o sinal de forma eficiente.

Montagem passo a passo:

A montagem foi dividida em duas unidades independentes (transmissor principal e receptor), construídas cada uma sobre duas protoboards.

1. Preparação dos módulos RF

Soldou-se um fio rígido de cobre de 17,3 cm no pad ANT de cada FS1000A e de cada XY-MK-5V. As pontas foram descascadas apenas nos últimos 5 mm para garantir um bom contacto elétrico. O restante do fio manteve o verniz isolante, que não afeta a irradiação.

2. Fixação e alimentação

Cada ESP 32 foi montado sobre um conjunto de duas protoboards unidas, formando uma base única para cada unidade. Todos os pinos ficaram acessíveis para facilitar as ligações. Cada ESP 32 possui o seu próprio par de módulos de rádio, um transmissor FS1000A e um receptor XY-MK-5V, totalizando quatro módulos no sistema. Esta disposição é necessária para que o protocolo Stop-and-Wait funcione corretamente: enquanto um módulo envia dados, o outro escuta os ACKs, e vice-versa. A alimentação dos módulos de rádio foi feita diretamente do pino VIN (5V) de cada ESP 32, que por sua vez estava a ser alimentado pela porta USB.

3. conexões do transmissor (ESP 32 A)

- GPIO2 - DATA do FS1000A (transmissor).
- GPIO34 - DATA do XY-MK-5V (receptor local).
- VCC do FS1000A (transmissor) - VIN (5 V) do ESP32.
- VCC de do XY-MK-5V (receptor local - VIN (5 V) do ESP32.
- GND do FS1000A (transmissor) - GND do ESP32.
- GND de do XY-MK-5V (receptor local - GND do ESP32.

4. Conexões do receptor (ESP 32 B)

- GPIO35 - DATA do XY-MK-5V (receptor principal).
- GPIO2 - DATA do FS1000A
- VCC do FS1000A (transmissor) - VIN (5 V) do ESP32.
- VCC de do XY-MK-5V (receptor local - VIN (5 V) do ESP32.
- GND do FS1000A (transmissor) - GND do ESP32.
- GND de do XY-MK-5V (receptor local - GND do ESP32.

5. Conexões do Display ST7735

- VCC - 3,3 V do ESP 32
- GND - GND do ESP 32
- SCK - GPIO18
- SDA (MOSI) - GPIO23
- A0 (DC) - GPIO16
- RESET - GPIO4
- CS - GPIO5
- LED (backlight) - 3,3 V,

Todas as ligações foram feitas com jumpers macho-fêmea, aproveitando os pinos fêmea da placa do display.

Protocolos de detecção de erros

Na camada de enlace implementou-se o protocolo Stop-and-Wait ARQ, que funciona da seguinte forma: o transmissor envia um quadro e fica aguardando uma confirmação (ACK) do receptor. Enquanto esse ACK não chega, o transmissor não envia o próximo quadro, daí o nome “para e espera”. Se ocorrer um timeout (tempo limite sem resposta), o mesmo quadro é retransmitido. Para evitar que quadros duplicados sejam aceitos, cada quadro de dados leva um número de sequência de 1 bit (0 ou 1), que se alterna a cada transmissão bem sucedida. O receptor, por sua vez, ignora quadros com número de sequência repetido, mas reenvia o ACK para garantir que o transmissor saiba que o dado foi entregue.

Para detectar erros de transmissão causados por ruído ou interferência, todos os quadros incluem um byte de verificação de redundância cíclica, o CRC-8. Utilizou-se o polinômio 0x07. O transmissor calcula o CRC sobre os campos do quadro (flag, tipo, sequência, tamanho e dados) e anexa no final. O receptor recalcula o CRC com os mesmos campos e compara com o valor recebido. Se houver qualquer diferença, o quadro é descartado, e o receptor simplesmente não envia ACK — o timeout no transmissor se encarregará de retransmitir.

Desenvolvimento

Transmissor (ESP 32 A)

O código do transmissor foi escrito na IDE Arduino e tem como tarefa principal receber uma mensagem de texto ou bitmap de uma imagem, dividi-la em pedaços, encapsular cada pedaço num quadro com proteção CRC e enviá-lo pelo rádio usando o protocolo Stop-and-Wait.

Estrutura do quadro

Cada quadro transmitido tem o seguinte formato:

- Flag (1 byte)
- Tipo (1 byte)
- Sequência (1 byte)
- Tamanho (1 byte)

- Dados (0 a 16 bytes)
- FCS (1 byte)

A flag foi definida como 0x7E e serve para delimitar o início do quadro. O campo Tipo indica se o quadro transporta texto (TIPO_TEXTO = 0x04), imagem (TIPO_IMAGEM = 0x01) ou se é uma confirmação (TIPO_ACK = 0x02). O campo Sequência carrega o bit de ordem (0 ou 1) exigido pelo Stop-and-Wait. Tamanho informa quantos bytes úteis estão presentes (máximo 16). O FCS (Frame Check Sequence) é o byte de CRC-8 calculado sobre os quatro primeiros campos mais os dados.

Cálculo do CRC-8

A função de cálculo realiza um XOR byte a byte, seguido de um laço de 8 iterações onde, se o bit mais significativo do registrador for 1, o valor é deslocado para a esquerda e recebe um XOR com o polinômio; caso contrário, apenas desloca. Esse método simula a divisão polinomial e produz o byte de verificação.

Trecho da função:

```
uint8_t crc8(uint8_t *data, uint8_t len) {
    uint8_t crc = 0;

    for (int i = 0; i < len; i++) {
        crc ^= data[i];

        for (int j = 0; j < 8; j++) {
            if (crc & 0x80)
                crc = (crc << 1) ^ 0x07;
            else
                crc <<= 1;
        }
    }

    return crc;
}
```

Envio dos bits

Como os módulos FS1000A não têm protocolo serial embutido, a modulação foi feita manualmente. A função `sendByte` transforma cada byte em uma sequência de 8 pulsos, com o bit mais significativo enviado primeiro (ordem MSB). Um bit 1 é transmitido como nível HIGH por 1000 microssegundos, seguido de LOW por 1000 microssegundos; um bit 0 é o contrário. Antes de cada quadro, um preâmbulo de 8 bytes 0xAA é enviado para estabilizar o AGC do receptor.

```
void sendBit(bool b) {
    digitalWrite(TX_PIN, b);
    delayMicroseconds(BIT_PERIOD);
}
void sendByte(uint8_t b) {
    digitalWrite(TX_PIN, HIGH);
    delayMicroseconds(BIT_PERIOD);
    for (int i = 0; i < 8; i++) {
        digitalWrite(TX_PIN, (b >> i) & 1);
        delayMicroseconds(BIT_PERIOD);
    }
    digitalWrite(TX_PIN, LOW);
    delayMicroseconds(BIT_PERIOD);
}
```

```
void sendPreamble() {
    for (int i = 0; i < 8; i++)
        sendByte(0xAA);
}
void sendFrame(uint8_t type, uint8_t seq, uint8_t *data, uint8_t len) {
    uint8_t buffer[22];
    buffer[0] = FLAG;
    buffer[1] = type;
    buffer[2] = seq;
    buffer[3] = len;
    for (int i = 0; i < len; i++)
        buffer[4 + i] = data[i];
    buffer[4 + len] = crc8(buffer, 4 + len);
    int total = 5 + len;
    sendPreamble();
    for (int i = 0; i < total; i++)
        sendByte(buffer[i]);
    digitalWrite(TX_PIN, LOW);
}
```

Protocolo Stop-and-Wait

O coração do transmissor é a função `sendData`, responsável por fragmentar o conteúdo e garantir que cada pedaço chegue corretamente. Ela recebe o tipo de dado (texto ou imagem), um ponteiro para o buffer e o comprimento total em bytes.

A rotina mantém um offset que percorre o buffer. Em cada iteração, calcula-se o tamanho do pedaço atual: se o que falta for maior que 16, usa-se 16; caso contrário, o que sobra. Os bytes desse pedaço são copiados para um array local (payload) e, em seguida, entra-se no laço de retransmissão. O quadro é enviado com `sendFrame`, que monta o pacote e o transmite. Depois, o transmissor aguarda um ACK válido por até 200 milissegundos, usando a função `readFrame`. Se o ACK chegar com o número de sequência correto, o offset avança e a sequência é invertida ($\text{seq} \wedge= 1$). Caso contrário, incrementa-se um contador de tentativas; ao fim de 5 tentativas sem sucesso, a transmissão é abortada.

Quando o comprimento total é múltiplo de 16, ainda é enviado um quadro adicional com “Tamanho” igual a zero, para sinalizar ao receptor que a imagem terminou. Esse mecanismo permite que o receptor distinga o fim da transmissão sem depender de um campo de comprimento total.

Trecho simplificado do laço principal:

```
while (!ack && tent < 5) {
    Serial.printf("TX tipo=%X seq=%d bloco=%d tam=%d tent=%d\n", type, seq, offset, chunk, tent + 1);
    sendFrame(type, seq, payload, chunk);
    digitalWrite(TX_PIN, LOW);
    delayMicroseconds(500);
    uint8_t rseq, rlen, rdata[16];
    uint8_t rtype = readFrame(rseq, rdata, rlen, 200);
    if (rtype == TYPE_ACK && rseq == seq)
        ack = true;
    else
        tent++;
}
if (!ack) {
    Serial.println("Falha TX");
    return false;
}
seq ^= 1;
offset += chunk;
}
```

Envio de texto e imagem

No código do transmissor, a escolha entre texto e imagem é feita por uma variável booleana `enviarTexto`. Quando verdadeira, o texto armazenado na string `mensagem` é passado para `sendData` com o tipo `TYPE_TEXT`. Quando falsa, o array `IMAGE_DATA` (512 bytes da imagem monocromática de 64 por 64 pixels) é enviado com o tipo `TYPE_IMAGE`. Em ambos os casos, a função `sendData` aplica a fragmentação e o Stop-and-Wait da mesma forma, mudando apenas o campo Tipo no quadro. O receptor, por sua vez, usa esse campo para decidir se acumula caracteres para exibir como string ou se monta o bitmap na memória.

```
const bool enviarTexto = true;
const char *msg = "Mensagem com mais de 16 bits para teste de pacotes 1234";

const uint8_t IMAGE_DATA[] PROGMEM = {
  //Bitmap da imagem aqui
};
```

Essa abordagem mantém a simplicidade do transmissor: toda a complexidade do controle de fluxo e detecção de erros fica concentrada em `sendData`, enquanto a escolha do conteúdo a enviar é feita por parâmetros facilmente alteráveis no início do código.

Receptor (ESP 32 B)

O Código do receptor também foi desenvolvido na IDE Arduino e é responsável por capturar os bits do módulo XY-MK-5V, remontar os quadros, verificar a integridade dos dados, executar a lógica do Stop-and-Wait e, conforme o tipo de conteúdo recebido, ou exibir um texto no display ou desenhar uma imagem monocromática.

Leitura dos bits e remontagem dos bytes

Como a transmissão é feita por bit-banging, o receptor precisa mostrar o pino de dados no momento correto. A função `readByte` implementa uma máquina de estados simples: aguarda uma borda de subida (start bit), depois insere um atraso de 1,5 período de bit para se posicionar exatamente no centro do primeiro bit de

dados. Em seguida, lê oito bits consecutivos e os monta em um byte, sempre na ordem LSB primeiro, a mesma usada pelo transmissor. Se dentro de 100 milissegundos nenhuma borda for detectada, a função retorna falso, o que evita que o processador fique travado eternamente à espera de um sinal que não chega.

Trecho da função de leitura:

```
bool readByte(uint8_t &byteRecebido) {
    uint32_t inicio = millis();
    while (digitalRead(RX_PIN) == LOW) {
        if (millis() - inicio > 100)
            return false;
    }
    delayMicroseconds(BIT_PERIOD + BIT_PERIOD / 2);
    uint8_t b = 0;
    for (int i = 0; i < 8; i++) {
        if (digitalRead(RX_PIN))
            b |= (1 << i);
        delayMicroseconds(BIT_PERIOD);
    }
    delayMicroseconds(BIT_PERIOD / 2);
    byteRecebido = b;
    return true;
}
```

Sincronismo de quadro e validação CRC

O laço principal do receptor chama readByte continuamente até que o byte lido coincida com a flag 0x7E. Quando isso acontece, o código lê os campos seguintes: Tipo, Sequência, Tamanho e os respectivos bytes de dados. De imediato é feita uma verificação de segurança: se o campo Tamanho for maior que 16, o quadro é descartado.

Em seguida, o receptor recalcula o CRC-8 sobre os mesmos campos usados pelo transmissor (flag, tipo, sequência, tamanho e dados). O polinômio é o mesmo 0x07, e a função crc8 é idêntica à do transmissor. Se o valor calculado não coincidir com o byte FCS recebido, o quadro é silenciosamente ignorado, o transmissor, ao não receber ACK, retransmite o quadro.

Lógica do Stop-and-Wait no receptor

Quando um quadro de dados (seja texto ou imagem) chega íntegro, o receptor executa a lógica de número de sequência. Se o campo Sequência for igual ao valor `expectedSeq` que o receptor aguarda, os dados são aceitos como novos e `expectedSeq` é invertido (`expectedSeq ^= 1`). Se a sequência for diferente da esperada, o quadro é tratado como duplicado (retransmissão), e os dados não são processados novamente, mas um ACK é enviado da mesma forma, para que o transmissor possa avançar.

Após processar o quadro (novo ou duplicado), o receptor envia imediatamente um quadro de ACK com o número de sequência que acabou de receber. Esse ACK é construído pela função `sendFrame`, que usa o transmissor FS1000A ligado ao GPIO2.

Trecho da lógica de sequência e envio de ACK:

```
if (seq == seqEsperado)
    seqEsperado ^= 1;
delay(1);
sendFrame(TYPE_ACK, seq, NULL, 0);
```

Tratamento do conteúdo: texto vs. imagem

O campo Tipo do quadro define o que fazer com os dados. Se for `TYPE_TEXT`, cada byte recebido é convertido para caractere e concatenado em uma string (`textBuffer`). Quando o tamanho do quadro é menor que 16, o receptor entende que aquele foi o último fragmento da mensagem. Nesse momento, a string acumulada é enviada para o display através da função `mostraTexto`.

Se o campo Tipo for `TYPE_IMAGE`, os bytes vão sendo copiados para um buffer de imagem de 512 posições (`imageBuffer`). Assim que o índice de bytes atinge 512, a função `mostraImagemMono` é chamada. Esta função percorre os 64 por 64 pixels e, para cada bit do buffer, decide se acende um pixel branco ou preto no display. O cálculo considera que cada byte do buffer representa 8 pixels consecutivos, com o

bit mais significativo correspondendo ao pixel mais à esquerda.

Trecho da função de desenho:

```
void mostraTexto(String msg) {  
    tft.fillScreen(ST77XX_BLACK);  
  
    tft.setCursor(0, 0);  
    tft.setTextColor(ST77XX_WHITE);  
    tft.setTextSize(1);  
  
    tft.println("Mensagem:");  
    tft.setTextColor(ST77XX_GREEN);  
  
    const int maxChars = 20;  
  
    while (msg.length() > maxChars) {  
        tft.println(msg.substring(0, maxChars));  
        msg = msg.substring(maxChars);  
    }  
  
    tft.println(msg);  
}
```

Interface com o display

O display ST7735 é inicializado com `tft.initR(INITR_144GREENTAB)`, valor descoberto por tentativa e erro durante a fase de testes. A rotação é configurada para o modo paisagem. Tanto o texto quanto a imagem são desenhados centralizados: o texto usa a função `println` com quebras manuais a cada 20 caracteres para não ultrapassar a largura da tela; a imagem calcula as coordenadas de forma que o bitmap de 64 por 64 fique exatamente no centro da área de 128 por 160 pixels.

Timeout e reset

Se o receptor ficar mais de cinco segundos sem receber um quadro válido, as variáveis de estado (expectedSeq, buffers de texto e imagem) são repostas para os valores iniciais. Isso evita que uma eventual dessincronização entre os dois lados se torne permanente. Quando a comunicação é retomada, ambos os lados voltam a operar normalmente a partir do primeiro quadro.

Essa arquitetura torna o receptor robusto e autônomo: ele não precisa de qualquer intervenção externa para se recuperar de falhas momentâneas de comunicação, característica essencial em enlaces de rádio sujeitos a interferências.

Resultados e discussão

Sucesso na transmissão de texto e imagem

O sistema mostrou-se capaz de transmitir tanto mensagens de texto curtas quanto a imagem monocromática de 64 por 64 píxeis. O protocolo Stop-and-Wait cumpriu a função de garantir a entrega, sendo possível observar no monitor série do transmissor o avanço correto dos offsets e a recepção dos ACKs. Quando um quadro era perdido por interferência, a retransmissão ocorria dentro do limite de cinco tentativas e a comunicação prosseguia sem intervenção manual. No lado do receptor, o display ST7735 exibiu o texto integral e desenhou a imagem centralizada, confirmando que a fragmentação e a remontagem dos dados estavam coerentes. A distância máxima de transmissão foi de cerca de 80 centímetros em espaço controlado, aberto e sem obstáculos.

Frustrações com falhas de comunicação

Durante os testes iniciais, os dois pares de módulos RF simplesmente não conversavam entre si. Mesmo com os códigos aparentemente corretos, o receptor permanecia mudo, sem detectar qualquer sinal do transmissor. Foram necessárias várias tentativas de reposicionamento físico dos módulos nas protoboards, variando a distância e a orientação das antenas, até que se conseguisse uma comunicação minimamente estável.

Troca dos jumpers artesanais por convencionais

Inicialmente, foram usados fios de cobre rígido descascados como jumpers. Apesar de funcionarem em prototipagem rápida, eles geravam maus contatos intermitentes, especialmente nos pinos de dados dos módulos de rádio, onde qualquer oscilação compromete a integridade do bit. A substituição por jumpers flexíveis com conectores fêmea padronizados eliminou as falhas intermitentes e trouxe muito mais estabilidade ao sistema. Esse ajuste simples foi determinante para que os testes pudessem ser repetidos de forma confiável.

Problemas na inicialização do display

O display ST7735 mostrou-se particularmente problemático na etapa de inicialização. Com o comando `tft.initR(INITR_BLACKTAB)`, a tela permanecia totalmente branca, embora a retroiluminação estivesse acesa. Foram testadas diversas opções de inicialização disponíveis na biblioteca Adafruit, e apenas `INITR_144GREENTAB` produziu o funcionamento correto. Uma vez encontrada a configuração adequada, o display operou sem novas dificuldades.

Possíveis melhorias futuras

Três frentes de melhoria foram identificadas para aumentar o alcance e a robustez do sistema. A primeira é a substituição das antenas atuais por outras de maior ganho, como antenas helicoidais, que melhorariam tanto a irradiação quanto a recepção. A segunda é fornecer uma alimentação externa de até 12V com maior capacidade de corrente exclusivamente para os módulos FS1000A, já que a linha VIN do ESP32 compartilha a corrente com o microcontrolador e pode sofrer quedas durante os picos de transmissão. A terceira é reduzir ainda mais a taxa de transmissão (aumentando `BIT_PERIOD` para valores como 2000 microssegundos), o que tornaria o sinal mais imune a ruídos e alargaria a margem de sincronismo entre transmissor e receptor.

Conclusão

O projeto conseguiu demonstrar com êxito a implementação de uma camada de enlace completa sobre módulos de rádio de 433 MHz, usando apenas bit-banging e código escrito manualmente, sem bibliotecas de terceiros para a comunicação RF. Os dois protocolos centrais , Stop-and-Wait ARQ e CRC-8, operaram de forma integrada, garantindo a detecção de erros e a confirmação de entrega mesmo em um canal ruidoso e sujeito a interferências. A estrutura de quadro customizada, com flag, tipo, sequência, tamanho e FCS, mostrou-se adequada para transportar tanto texto quanto imagens fragmentadas.

A distância máxima alcançada, de 80 centímetros, ficou abaixo do esperado para módulos de 433 MHz, mas revelou a importância de elementos muitas vezes negligenciados em prototipagem, como a qualidade das antenas e a estabilidade das ligações físicas. A soldagem de antenas de quarto de onda e a substituição de jumpers artesanais por cabos flexíveis com conectores apropriados foram intervenções simples que transformaram um sistema praticamente não funcional em um enlace confiável.

As dificuldades de sincronismo e a tendência do receptor a travar por causa do ruído evidenciaram a necessidade de mecanismos de recuperação autônoma. A introdução de um timeout de cinco segundos para reset das variáveis de estado foi a solução que confere robustez ao sistema, permitindo que ele se recuperasse sozinho após falhas momentâneas, sem exigir intervenção manual.

Em síntese, o trabalho cumpriu os objetivos propostos: estabeleceu um enlace bidirecional simulado com confirmação de quadros, validou a fragmentação e remontagem de dados, e integrou uma interface visual para exibição do conteúdo recebido. As limitações de alcance e as dificuldades enfrentadas serviram como aprendizado prático sobre os desafios reais da comunicação sem fios, apontando claramente os caminhos para melhorias futuras, como antenas de maior ganho, alimentação dedicada para os módulos de transmissão e redução da taxa de bits.

Referências

Marlon Nardi. **Comunicação sem fio entre Arduinos usando módulos RF TX e RX 433MHz.** 2021. 1 vídeo (10 min). Disponível em: <https://www.youtube.com/watch?v=B9gvxDHW65A>. Acesso em: 10 jun. 2026.

Shinobi Artss . **Como Usar Display TFT 1.8" ST7735 no ESP32.** 2026. 1 vídeo (8 min). Disponível em: <https://www.youtube.com/watch?v=8oz1ASiu5g>. Acesso em: 10 jun. 2026.

Bit a Bit . **Arduino - Usando o módulo rádio frequência.** 2022. 1 vídeo (6 min). Disponível em: https://youtu.be/DVq_3fLQyJk. Acesso em: 10 jun. 2026.

ROBOTISTAN. **433 MHz RF transmitter and receiver module – datasheet. [S.l.], [202-].** 4 p. Disponível em: <https://www.robotistan.com/Data/EditorFiles/433MHZ.PDF>. Acesso em: 11 jun. 2026.

KUROSE, J. F.; ROSS, K. W. **Redes de computadores e a internet: uma abordagem top-down.** 8. ed. São Paulo: Pearson, 2021. Cap. 3 (Stop-and-Wait ARQ).

TANENBAUM, A. S.; FEAMSTER, N.; WETHERALL, D. J. **Redes de computadores.** 6. ed. Porto Alegre: Bookman, 2021. Seção 3.3 (Protocolos de janela deslizante e Stop-and-Wait).

WILLIAMS, R. N. **A painless guide to CRC error detection algorithms.** Rocksoft, 1993. Disponível em: https://zlib.net/crc_v3.txt. Acesso em: 12 jun. 2026.